

Name: - Shubham Das

S. A. P I. D: - 590033500

Date: - 10/09/2026

Programme: - B.TECH (CSE)

Assignments: - Write a menu-driven C programme using switch-case to perform digit-based operations on a positive integer. Define one separate user-defined function for each menu option, with void return type and with a simple function call (without call by value/reference). Each function should read the required input, process it, and display the result. Prepare the algorithm and test case table. Prepare the algorithm and test case table. Then write, compile, execute, and test the C programme against the prepared use cases.

Generate the largest possible number using its digits. Generate the smallest possible number using its digits. Swap its first and last digits. Check whether it is a palindrome number.

Part 1: - Thorough understanding of the problem statement

Input: - 'choice' (Menu option selection, Variable type: 'int', Unit: unitless integer)
'Num' (Positive integer processed locally inside each function, Variable type: 'long int', Unit: unitless integer)

Output: - Option 1: - Largest number possible from the digits (Variable type: 'long int')

Option 2: - Smallest number possible from the digits (Variable type: 'long int')

Option 3: - Number with swapped first and last digits (Variable type: 'long int')

Option 4: - Palindrome status ("Palindrome" or "Not Palindrome", text string)

Formula (in C logic)

Count Frequencies (Options 1 & 2): 'freq[digit]++'

Construct Largest: Loop 'i' from 9 down to 0, 'largest = largest * 10 + i;'

Construct Smallest: Find first non-zero digit, place it, then loop 'i' from 0 to 9:
'smallest = smallest * 10 + i'

Swap First/Last: 'first = num/div; last = num % 10; middle = (num 5 % div) / 10;
swapped = (last * div) + (middle * 10) + first;

Reverse for Palindrome: rev = rev * 10 + (temp % 10); temp /= 10;

Part 2:- Writing and evaluating the algorithm

Step 1: Start

Step 2: Define four functions: 'generateLargest()', 'generateSmallest()', 'swapFirstLast()', and 'checkPalindrome()'. All with 'void' return types and no parameters.

Step 3: - Inside 'main()', declare 'choice'. Loop the menu until the user chooses to exit.

Step 4: - Display the menu and read 'choice'.

Step 5: - Use a 'switch case' to call the respective function based on 'choice' (no arguments passed).

Step 6 (generateLargest): - Prompt for 'num'. Extract digits and count frequencies in 'freq[10]'. Iterate from 9 to 0. For each count, append the digit to 'largest'. Print 'largest'.

Step 7 (generateSmallest): - Prompt for 'num'. Extract digits into 'freq[10]'. Find the smallest non-zero digit to be the leading digit. Then iterate from 0 to 9, appending remaining digits to 'Smallest'. Print 'smallest'.

Step 8 (swapFirstLast): - Prompt for 'num'. Calculate a 'divisor' (e.g., 10000 for 12345). Extract 'first', 'last', and 'middle' sections. Recombine as '(last * divisor) + (middle * 10) + first'. Print the result.

Step 9 (checkPalindrome): - Prompt for 'num'. Reverse the number mathematically. If 'reversed == original', print 'Palindrome', else "Not Palindrome".

Part 3: - Writing and compiling the C programme

kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox

File Machine View Input Devices Help

GNU nano 9.0

```
#include <stdio.h>

// Function prototypes
void generateLargest(void);
void generateSmallest(void);
void swapFirstLast(void);
void checkPalindrome(void);

int main() {
    int choice;

    do {
        printf("\n===== \n");
        printf("          Digit Operations Menu          \n");
        printf("===== \n");
        printf("1. Generate the largest possible number\n");
        printf("2. Generate the smallest possible number\n");
        printf("3. Swap first and last digits\n");
        printf("4. Check whether it is a palindrome\n");
        printf("5. Exit\n");
        printf("Enter your choice (1-5): ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                generateLargest();
                break;
            case 2:
                generateSmallest();
                break;
            case 3:
                swapFirstLast();
                break;
            case 4:
                checkPalindrome();
                break;
            case 5:
                printf("Exiting program. Goodbye!\n");
                break;
            default:
                printf("Error: Invalid choice. Please select 1-5.\n");
        }
    } while (choice != 5);

    return 0;
}

// Option 1: Generate Largest Number
```

Help Write Out Where Is Cut Execute
Exit Read File Replace Paste Justify

kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox

File Machine View Input Devices Help

1 2 3 4 5 2

Session Actions Edit View Help

GNU nano 9.0

```
// Option 1: Generate Largest Number
void generateLargest(void) {
    long int num, temp, largest = 0;
    int freq[10] = {0}, i, j;

    printf("\nEnter a positive integer: ");
    scanf("%ld", &num);

    if (num <= 0) { printf("Error: Positive integers only.\n"); return; }

    temp = num;
    while (temp > 0) {
        freq[temp % 10]++;
        temp /= 10;
    }

    for (i = 9; i >= 0; i--) {
        for (j = 0; j < freq[i]; j++) {
            largest = largest * 10 + i;
        }
    }
    printf("Original Number: %ld\nLargest Possible Number: %ld\n", num, largest);
}

// Option 2: Generate Smallest Number
void generateSmallest(void) {
    long int num, temp, smallest = 0;
    int freq[10] = {0}, i, j;

    printf("\nEnter a positive integer: ");
    scanf("%ld", &num);

    if (num <= 0) { printf("Error: Positive integers only.\n"); return; }

    temp = num;
    while (temp > 0) {
        freq[temp % 10]++;
        temp /= 10;
    }

    // Find the first non-zero digit to prevent leading zeros
    for (i = 1; i <= 9; i++) {
        if (freq[i] > 0) {
            smallest = i;
            freq[i]--;
            break;
        }
    }
}
```

Help Write Out Where Is Cut Execute
Exit Read File Replace Paste Justify

kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox

File Machine View Input Devices Help

1 2 3 4

kali@kali: ~

GNU nano 9.8 menu_operations.c *

```
}

// Append the rest of the digits including zeros
for (i = 0; i <= 9; i++) {
    for (j = 0; j < freq[i]; j++) {
        smallest = smallest * 10 + i;
    }
}
printf("Original Number: %ld\nSmallest Possible Number: %ld\n", num, smallest);
}

// Option 3: Swap First and Last Digits
void swapFirstLast(void) {
    long int num, temp, swapped;
    long int divisor = 1;
    int first, last, middle;

    printf("\nEnter a positive integer: ");
    scanf("%ld", &num);

    if (num <= 0) { printf("Error: Positive integers only.\n"); return; }
    if (num < 10) {
        printf("Original Number: %ld\nSwapped Number: %ld (Single digit remains unchanged)\n", num, num);
        return;
    }

    temp = num;
    // Find the divisor to extract the first digit
    while (temp >= 10) {
        divisor *= 10;
        temp /= 10;
    }

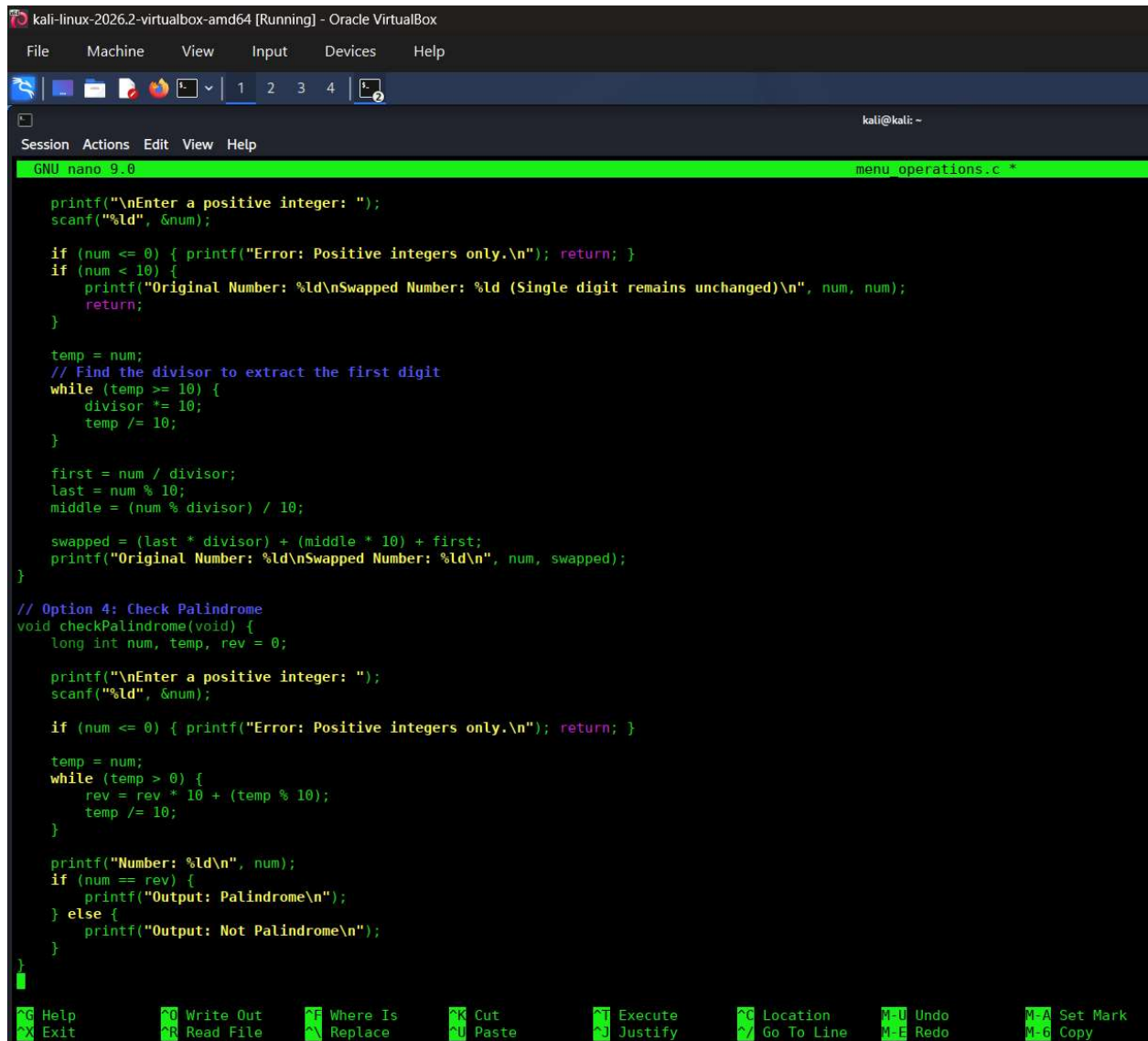
    first = num / divisor;
    last = num % 10;
    middle = (num % divisor) / 10;

    swapped = (last * divisor) + (middle * 10) + first;
    printf("Original Number: %ld\nSwapped Number: %ld\n", num, swapped);
}

// Option 4: Check Palindrome
void checkPalindrome(void) {
    long int num, temp, rev = 0;

    printf("\nEnter a positive integer: ");
    scanf("%ld", &num);
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^G Location M-U Undo M-A Set Mark
^X Exit ^R Read File ^N Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo M-G Copy



```
kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
1 2 3 4
kali@kali: ~
GNU nano 9.8 menu_operations.c *

printf("\nEnter a positive integer: ");
scanf("%ld", &num);

if (num <= 0) { printf("Error: Positive integers only.\n"); return; }
if (num < 10) {
    printf("Original Number: %ld\nSwapped Number: %ld (Single digit remains unchanged)\n", num, num);
    return;
}

temp = num;
// Find the divisor to extract the first digit
while (temp >= 10) {
    divisor *= 10;
    temp /= 10;
}

first = num / divisor;
last = num % 10;
middle = (num % divisor) / 10;

swapped = (last * divisor) + (middle * 10) + first;
printf("Original Number: %ld\nSwapped Number: %ld\n", num, swapped);
}

// Option 4: Check Palindrome
void checkPalindrome(void) {
    long int num, temp, rev = 0;

    printf("\nEnter a positive integer: ");
    scanf("%ld", &num);

    if (num <= 0) { printf("Error: Positive integers only.\n"); return; }

    temp = num;
    while (temp > 0) {
        rev = rev * 10 + (temp % 10);
        temp /= 10;
    }

    printf("Number: %ld\n", num);
    if (num == rev) {
        printf("Output: Palindrome\n");
    } else {
        printf("Output: Not Palindrome\n");
    }
}

}
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^J Execute    ^G Location   ^U Undo       ^A Set Mark
^X Exit      ^R Read File  ^N Replace    ^U Paste      ^D Justify    ^_ Go To Line  ^E Redo       ^B Copy
```

Part 4: - Running and executing the algorithm on a platform (Kali Linux VM)

Step 1: - Open the terminal

Step 2: - Create the source code

Step 3: - Write and Save Code

Step 4: - Compile the programme

Step 5: - Run the Binary Executable programme

```
(kali㉿kali)-[~]
```

```
$ nano menu_operations.c
```

```
(kali㉿kali)-[~]
```

```
$ gcc menu_operations.c -o menu_operations
```

```
(kali㉿kali)-[~]
```

```
$ ./menu_operations
```

```
=====
Digit Operations Menu
=====
```

1. Generate the largest possible number
2. Generate the smallest possible number
3. Swap first and last digits
4. Check whether it is a palindrome
5. Exit

```
Enter your choice (1-5): 1
```

```
Enter a positive integer: 48291
```

```
Original Number: 48291
```

```
Largest Possible Number: 98421
```

```
=====
Digit Operations Menu
=====
```

1. Generate the largest possible number
2. Generate the smallest possible number
3. Swap first and last digits
4. Check whether it is a palindrome
5. Exit

```
Enter your choice (1-5): 2
```

```
Enter a positive integer: 4020
```

```
Original Number: 4020
```

```
Smallest Possible Number: 2004
```

```
=====
Digit Operations Menu
=====
```

1. Generate the largest possible number
2. Generate the smallest possible number
3. Swap first and last digits
4. Check whether it is a palindrome

=====

Digit Operations Menu

=====

1. Generate the largest possible number
2. Generate the smallest possible number
3. Swap first and last digits
4. Check whether it is a palindrome
5. Exit

Enter your choice (1-5): 3

Enter a positive integer: 12345

Original Number: 12345

Swapped Number: 52341

=====

Digit Operations Menu

=====

1. Generate the largest possible number
2. Generate the smallest possible number
3. Swap first and last digits
4. Check whether it is a palindrome
5. Exit

Enter your choice (1-5): 4

Enter a positive integer: 12321

Number: 12321

Output: Palindrome

=====

Digit Operations Menu

=====

1. Generate the largest possible number
2. Generate the smallest possible number
3. Swap first and last digits
4. Check whether it is a palindrome
5. Exit

Enter your choice (1-5): 3

Enter a positive integer: 8

Original Number: 8

Swapped Number: 8 (Single digit remains unchanged)

=====

```
kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
[Icons] | 1 2 3 4 | [Terminal Icon] 2

Session Actions Edit View Help
=====
Digit Operations Menu
=====
1. Generate the largest possible number
2. Generate the smallest possible number
3. Swap first and last digits
4. Check whether it is a palindrome
5. Exit
Enter your choice (1-5): 4

Enter a positive integer: 12321
Number: 12321
Output: Palindrome

=====
Digit Operations Menu
=====
1. Generate the largest possible number
2. Generate the smallest possible number
3. Swap first and last digits
4. Check whether it is a palindrome
5. Exit
Enter your choice (1-5): 3

Enter a positive integer: 8
Original Number: 8
Swapped Number: 8 (Single digit remains unchanged)

=====
Digit Operations Menu
=====
1. Generate the largest possible number
2. Generate the smallest possible number
3. Swap first and last digits
4. Check whether it is a palindrome
5. Exit
Enter your choice (1-5): 4

Enter a positive integer: 123
Number: 123
Output: Not Palindrome

=====
Digit Operations Menu
=====
```

Part 5:- Standard test table

TEST CASE	SELECTED OPTION	INPUT ('num')	EXPECTED OUTPUT	ACTUAL OUTPUT	RESULT (PASS/FAIL)
1	1 (Largest)	48291	Largest: 98421	Largest: 98421	PASS
2	2 (Smallest)	4020	Smallest: 2004	Smallest: 2004	PASS
3	3 (Swap)	12345	Swapped: 52341	Swapped: 52341	PASS
4	4 (Palindrome)	12321	Output: Palindrome	Output: Palindrome	PASS
5	3 (Swap)	8	Swapped: 8	Swapped: 8	PASS
6	4 (Palindrome)	123	Output: Not Palindrome	Output: Not Palindrome	PASS

Part 6: - Evaluation of the algorithm

The menu loop creates an efficient and re-usable interactive terminal session until gracefully exited by the user.